

Proyecto CS 2031 - DBP

La segunda fase del proyecto de Desarrollo Basado en Plataformas (DBP) constituye un elemento esencial del programa académico, representando el 20% de la nota final. Su importancia radica en la integración y aplicación práctica de las competencias adquiridas en el curso, incluyendo el uso de diversas herramientas, conceptos y tecnologías. Este proyecto no solo evalúa el conocimiento técnico, sino también habilidades como el trabajo en equipo, la innovación y la presentación efectiva de ideas.

Esta entrega corresponde a la culminación del desarrollo front-end (web y mobile), el deployment integral de la aplicación, y la presentación final del proyecto. A continuación, se presenta la documentación detallada que abarca los requisitos, entregables y criterios de evaluación para esta fase final.

Fechas Importantes

- **Entrega del Proyecto:** Martes 28 de junio a las 11:59 p.m. (Semana 14)
- **Presentaciones en Laboratorio:** Semana 15
- **Anuncio de Finalistas:** Posterior a las presentaciones de laboratorio
- **Presentación Final (Finalistas):** Auditorio - Última semana del curso
- **Premio Tim Berners-Lee:** Solo los equipos finalistas tienen posibilidad de participar

Entregables

Para la entrega de la segunda parte del proyecto se considerarán dos factores principales: el código entregado y la presentación durante la clase.

Código

El código debe estar hosteado en el repositorio de GitHub del equipo y cumplir con todos los puntos indicados en la rúbrica proporcionada. Debe incluir:

- **Backend:** API REST completamente funcional (desarrollada en la primera fase)
- **Frontend Web:** Aplicación web desarrollada en React
- **Frontend Mobile:** Aplicación móvil desarrollada en React Native
- **Deployment:** Todos los componentes desplegados y accesibles públicamente

Presentación

La presentación será un pitch tecnológico al estilo de las presentaciones de empresas tecnológicas (similar a Apple), con las siguientes características:

- **Duración:** 20 minutos
- **Formato:** Presentación profesional (PPT, Canva, etc.) + Demo en vivo
- **Estructura:** Gancho inicial, planteamiento del problema, presentación de la solución, arquitectura técnica, demo funcional
- **Evaluación:** Se incluirá una sesión de preguntas sobre el funcionamiento del proyecto y fragmentos específicos del código

Recomendaciones:

- Iniciar con un gancho que resalte el problema a resolver
 - Presentar la problemática de manera clara y concisa
 - Tener datos preparados para el demo (evitar procesos largos durante la presentación)
 - Practicar previamente para asegurar fluidez y evitar problemas técnicos
 - Preparar un flujo que demuestra claramente cómo la aplicación soluciona el problema principal
-

Rúbrica de Evaluación

1. Exposición del Proyecto (5 puntos) Individual

4 puntos: Muestra dominio completo del proyecto, responde todas las preguntas del docente tanto teóricas como técnicas. La PPT está estructurada, clara y profesional. El demo muestra todas las funciones prometidas sin problemas técnicos. La presentación es dinámica, comienza con un gancho atractivo y mantiene la atención del público. Explica la problemática de manera clara y concisa.

3 puntos: Muestra buen dominio del proyecto, responde la mayoría de las preguntas del docente con precisión. La PPT está bien estructurada y el demo muestra la mayoría de las funciones prometidas, con pocos problemas técnicos. La presentación mantiene el interés del público la mayor parte del tiempo y explica la problemática adecuadamente.

2 puntos: Muestra dominio parcial del proyecto, responde algunas preguntas del docente con ciertas dudas. La PPT y el demo muestran algunas funciones prometidas, pero con varios problemas técnicos. El gancho inicial es poco efectivo y la presentación pierde el interés del público en varios momentos. La explicación de la problemática es confusa o incompleta.

1 punto: Muestra escaso dominio del proyecto, responde muy pocas preguntas del docente, con errores graves. La PPT está mal diseñada y el demo en vivo muestra pocas funciones prometidas, con problemas técnicos importantes. No tiene un gancho y la presentación es monótona y aburrida. No se explica la problemática o se hace de manera incorrecta.

0 puntos: No es capaz de responder ninguna de las preguntas del docente, demostrando desconocimiento total del proyecto. No se presenta PPT o el demo no funciona.

2. Mobile (5 puntos)

2.1 Integración con Backend y Consumo de API (1.5 puntos)

- **1.5 puntos:** La aplicación consume exclusivamente el backend desarrollado como core del proyecto. Todas las funcionalidades principales dependen de la API REST. Implementa correctamente los métodos HTTP (GET, POST, PUT, DELETE, PATCH) según corresponda. Maneja tokens JWT de manera segura usando el SDK de Expo (SecureStore). Los requests incluyen headers apropiados (Authorization, Content-Type). Implementa interceptores para manejo centralizado de autenticación. No hay dependencia de datos en memoria para funcionalidades core.
- **1.2 puntos:** Consume el backend como core, pero tiene dependencia menor de datos en memoria para funcionalidades secundarias. Implementa correctamente la mayoría de métodos HTTP. Maneja tokens de manera segura, aunque no usa completamente el SDK de Expo. Los headers están presentes en la mayoría de requests.
- **0.9 puntos:** Consume el backend, pero algunas funcionalidades core dependen de datos en memoria o APIs de terceros. Implementa métodos HTTP básicos. El manejo de tokens es funcional pero no completamente seguro. Headers inconsistentes en algunos requests.
- **0.5 puntos:** Consume parcialmente el backend. Varias funcionalidades core dependen de datos en memoria. Manejo de tokens básico o inseguro. Falta de headers apropiados en muchos requests.
- **0 puntos:** No consume el backend desarrollado o la integración es completamente deficiente.

2.2 Arquitectura y Separación de Responsabilidades (1 punto)

- **1 punto:** Arquitectura claramente modular con separación completa de capas: servicios/API (axios configurado con baseURL, interceptores), lógica de negocio (custom hooks, context API), y UI (componentes presentacionales). Usa React Native como framework principal. Implementa custom hooks para lógica reusable. Context API o state management para estado global. Componentes funcionales con hooks (useState, useEffect, useContext, useReducer). Código limpio y organizado en carpetas lógicas (screens, components, services, hooks, contexts, utils). Nombres descriptivos en inglés para variables, funciones y componentes.
- **0.8 puntos:** Arquitectura modular con buena separación de capas. Usa React Native. Implementa algunos custom hooks. Usa Context API o state management básico. Buenos componentes funcionales con hooks. Organización de carpetas presente pero mejorable. Nombres descriptivos en su mayoría.

- **0.6 puntos:** Arquitectura funcional pero con separación de responsabilidades mejorable. Lógica mezclada entre capas. Uso básico de hooks. State management limitado. Organización de carpetas básica. Algunos nombres poco descriptivos.
- **0.3 puntos:** Arquitectura poco modular. Separación de responsabilidades confusa. Hooks mal implementados o escasos. Sin state management apropiado. Organización deficiente. Nombres genéricos o inconsistentes.
- **0 puntos:** Sin arquitectura clara. Código desorganizado. No usa React Native apropiadamente.

2.3 Sensores y APIs Externas (1.5 puntos)

- **1.5 puntos:** Implementa al menos dos sensores del dispositivo que aportan valor real al proyecto (cámara para fotos/QR, GPS para ubicación, acelerómetro para gestos, giroscopio, micrófono, etc.). Los sensores están integrados correctamente usando expo-camera, expo-location, u otras librerías apropiadas. Consume al menos una API externa relevante al proyecto (mapas, clima, pagos, etc.). Maneja permisos correctamente con mensajes claros al usuario. Implementa fallbacks cuando los sensores no están disponibles.
- **1 punto:** Implementa dos sensores relevantes con buena integración. Consume API externa. Maneja permisos adecuadamente. Tiene algunos fallbacks.
- **0.8 puntos:** Implementa un sensor relevante correctamente. Puede consumir API externa básica. Manejo de permisos presente pero mejorable. Fallbacks limitados.
- **0.4 puntos:** Implementa un sensor de manera básica. API externa no relevante o mal integrada. Manejo de permisos deficiente. Sin fallbacks.
- **0 puntos:** No implementa sensores o están completamente mal integrados. No consume APIs externas relevantes.

2.4 Manejo de Errores y Estados de Carga (1 punto)

- **1 punto:** Implementa manejo completo de errores con try-catch en todas las llamadas API. Muestra mensajes de error amigables y específicos al usuario (no técnicos). Implementa estados de carga (loading) con indicadores visuales (spinners, skeletons). Maneja errores de red, timeouts, errores 4xx y 5xx apropiadamente. Implementa retry logic para errores recuperables. Muestra feedback visual para acciones exitosas (confirmaciones, toasts). Valida datos antes de enviar requests. Maneja casos edge (listas vacías, sin conexión, etc.) con mensajes apropiados.
- **0.8 puntos:** Buen manejo de errores con try-catch en la mayoría de llamadas. Mensajes de error amigables. Estados de carga implementados. Maneja errores comunes. Feedback visual para acciones principales. Validaciones presentes.
- **0.6 puntos:** Manejo básico de errores. Algunos mensajes técnicos aún visibles. Estados de carga en pantallas principales. Maneja algunos tipos de error. Feedback visual limitado. Validaciones básicas.
- **0.3 puntos:** Manejo de errores inconsistente. Mensajes de error técnicos. Estados de carga escasos. No maneja todos los tipos de error. Poco feedback visual. Validaciones insuficientes.

- **0 puntos:** Sin manejo de errores. Sin estados de carga. Sin validaciones. Sin feedback visual.

3. Web (5 puntos)

3.1 Integración con Backend y Consumo de API (2 puntos)

- **2 puntos:** La aplicación consume exclusivamente el backend desarrollado como core del proyecto. Implementa una capa de cliente bien abstraída usando axios con configuración centralizada (baseUrl, headers por defecto, interceptores). Todas las funcionalidades principales dependen de la API REST. Implementa correctamente los métodos HTTP (GET, POST, PUT, DELETE, PATCH). Maneja tokens JWT almacenados de manera segura (sin localStorage en artifacts, pero usando métodos seguros en implementación real). Los interceptores manejan autenticación, refresh tokens, y errores globales. Implementa request cancellation con AbortController para evitar memory leaks. No hay dependencia de datos hardcodeados para funcionalidades core.
- **1.5 puntos:** Consume el backend como core con buena abstracción de axios. Configuración centralizada presente. Implementa la mayoría de métodos HTTP correctamente. Manejo de tokens funcional. Interceptores para autenticación y errores básicos. Minimal dependencia de datos hardcodeados.
- **0.9 puntos:** Consume el backend pero la abstracción de axios es mejorable. Configuración parcialmente centralizada. Métodos HTTP básicos implementados. Manejo de tokens presente. Interceptores limitados. Alguna dependencia de datos hardcodeados.
- **0.5 puntos:** Consume parcialmente el backend. Abstracción deficiente de axios. Configuración no centralizada. Manejo de tokens básico o inseguro. Sin interceptores apropiados. Dependencia significativa de datos hardcodeados.
- **0 puntos:** No consume el backend desarrollado o la integración es completamente deficiente.

3.2 Componentes y Arquitectura React (1 punto)

- **1 punto:** Implementa más de 10 componentes diferenciados y genuinamente reutilizables. Usa React como librería principal. Separación clara entre componentes presentacionales y contenedores. Implementa custom hooks para lógica reutilizable (useAuth, useFetch, useForm, etc.). Usa React hooks eficientemente (useState, useEffect, useContext, useReducer, useMemo, useCallback). Context API o state management (Redux, Zustand) para estado global. Arquitectura modular con carpetas organizadas (components, pages, hooks, services, contexts, utils). Componentes pequeños y enfocados (SRP). Props bien tipadas con PropTypes o TypeScript. Nombres descriptivos en inglés. Código limpio sin comentarios innecesarios.
- **0.8 puntos:** Implementa entre 8-10 componentes reutilizables. Usa React correctamente. Buena separación de componentes. Implementa algunos custom

hooks. Usa hooks apropiadamente. State management presente. Buena organización de carpetas. Componentes enfocados. Nombres descriptivos.

- **0.6 puntos:** Implementa entre 6-7 componentes. Algunos genuinamente reutilizables. Usa React. Separación de componentes básica. Pocos custom hooks. Uso básico de hooks. State management limitado. Organización mejorable. Algunos componentes grandes.
- **0.3 puntos:** Implementa 5 o menos componentes. Reutilización limitada. Separación confusa. Sin custom hooks. Hooks mal implementados. Sin state management apropiado. Organización deficiente. Componentes monolíticos.
- **0 puntos:** No usa React apropiadamente. Sin componentes reutilizables. Código desorganizado.

3.3 Routing y Navegación (1 punto)

- **1 punto:** Implementa React Router (o similar) con rutas dinámicas usando parámetros (:id) y query params (?page=1&search=term). Rutas protegidas con componentes de autenticación (PrivateRoute). Lazy loading de componentes con React.lazy y Suspense. Navegación programática con useNavigate. Manejo de rutas 404. Breadcrumbs o indicadores de ubicación. Historial de navegación preservado. Rutas organizadas en archivo/configuración centralizada.
- **0.8 puntos:** Implementa React Router con rutas dinámicas y query params. Rutas protegidas. Lazy loading básico. Navegación programática. Manejo de 404. Configuración organizada.
- **0.6 puntos:** Implementa React Router con rutas dinámicas. Algunas rutas protegidas. Sin lazy loading. Navegación básica. Manejo de 404 limitado.
- **0.3 puntos:** Routing básico. Rutas estáticas principalmente. Sin protección de rutas. Sin manejo de 404.
- **0 puntos:** Sin routing implementado o completamente deficiente.

3.4 Manejo de Errores, Estados de Carga y Validaciones (1 punto)

- **1 punto:** Implementa manejo exhaustivo de errores con try-catch en todas las llamadas API. Error boundaries para capturar errores de renderizado. Mensajes de error específicos y amigables al usuario (no técnicos). Estados de carga (loading) con indicadores visuales apropiados (spinners, skeletons, progress bars). Maneja todos los códigos de estado HTTP (400, 401, 403, 404, 409, 500) con respuestas diferenciadas. Implementa validaciones client-side con librerías (Formik, React Hook Form) o custom. Validación en tiempo real con feedback visual. Muestra feedback para acciones exitosas (toasts, alerts, confirmaciones). Maneja casos edge (listas vacías, sin resultados, sin conexión) con mensajes y visuales apropiados. Deshabilita botones durante requests para evitar doble-submit. Implementa retry logic para errores recuperables.
- **0.8 puntos:** Buen manejo de errores con try-catch. Mensajes amigables. Estados de carga implementados. Maneja códigos HTTP principales. Validaciones client-side presentes. Feedback visual para acciones importantes. Maneja casos edge comunes. Previene doble-submit.

- **0.6 puntos:** Manejo básico de errores. Algunos mensajes técnicos visibles. Estados de carga en páginas principales. Maneja algunos códigos HTTP. Validaciones básicas. Feedback limitado. Algunos casos edge manejados.
- **0.3 puntos:** Manejo inconsistente de errores. Mensajes técnicos. Estados de carga escasos. No diferencia códigos HTTP. Validaciones insuficientes. Poco feedback. Casos edge no manejados. Sin prevención de doble-submit.
- **0 puntos:** Sin manejo de errores. Sin estados de carga. Sin validaciones. Sin feedback visual.

4. Web - UI/UX (2 puntos)

4.1 Diseño Visual y Estilización (0.5 punto)

- **0.5 punto:** Diseño atractivo, moderno y profesional. Usa framework CSS (Tailwind, Material-UI, Ant Design, Chakra UI) o CSS/SCSS bien organizado. Componentes consistentemente estilizados. Paleta de colores coherente y apropiada. Tipografía legible y consistente. Espaciado y alineación apropiados. Responsive design funcional en múltiples tamaños de pantalla (mobile, tablet, desktop). Imágenes optimizadas. Iconos consistentes (Font Awesome, Material Icons, etc.). Transiciones y animaciones sutiles que mejoran UX. Dark mode o temas alternativos (bonus).
- **0.3 puntos:** Diseño atractivo y profesional. Usa framework CSS o estilos bien organizados. Componentes bien estilizados. Paleta coherente. Tipografía apropiada. Responsive design funcional. Iconos consistentes. Algunas animaciones.
- **0.2 puntos:** Diseño aceptable. Usa algún framework CSS o estilos básicos. Estilización presente pero inconsistente. Responsive limitado. Iconos presentes. Pocas animaciones.
- **0.1 puntos:** Diseño básico. Estilización mínima. No usa frameworks CSS apropiadamente. No responsive. Iconos escasos. Sin animaciones.
- **0 puntos:** Sin diseño aparente. Estilos completamente básicos o ausentes.

4.2 Experiencia de Usuario (0.5 punto)

- **0.5 punto:** Navegación intuitiva y lógica. Flujos de usuario claros y sin fricción. Formularios con labels claros, placeholders apropiados, y mensajes de ayuda. Confirmaciones para acciones destructivas. Breadcrumbs o indicadores de progreso en flujos multi-paso. Búsqueda funcional con debounce. Filtros y ordenamiento en listados. Accesibilidad básica (contraste adecuado, alt text en imágenes, navegación por teclado). Loading states que no bloquean la UI innecesariamente. Mensajes de éxito/error contextuales. Tooltips o hints donde sea necesario. Manejo elegante de estados vacíos con call-to-actions.
- **0.3 puntos:** Navegación intuitiva. Flujos claros. Formularios bien diseñados. Confirmaciones presentes. Indicadores de progreso. Búsqueda funcional. Algunos filtros. Accesibilidad básica. Buenos loading states. Mensajes contextuales. Estados vacíos manejados.

- **0.2 puntos:** Navegación funcional. Flujos comprensibles. Formularios básicos. Algunas confirmaciones. Búsqueda simple. Filtros limitados. Accesibilidad mínima. Loading states básicos. Mensajes genéricos.
- **0.1 puntos:** Navegación confusa. Flujos poco claros. Formularios básicos sin ayuda. Sin confirmaciones. Sin búsqueda. Sin filtros. Sin accesibilidad. Loading states deficientes. Mensajes pobres.
- **0 puntos:** UX completamente deficiente. Navegación confusa. Sin flujos claros.

4.3 Vistas y Funcionalidades (0.5 puntos)

- **0.5 puntos:** Implementa más de 4 vistas diferenciadas (login, registro, dashboard, perfil, feed, listados, detalle, configuraciones, etc.). Cada vista tiene propósito claro y diseño apropiado. Dashboard con información relevante y visualizada apropiadamente. Vistas de listado con paginación funcional consumiendo endpoints paginados del backend. Vista de detalle con información completa. Formularios de creación/edición bien estructurados.
- **0.3 puntos:** Implementa 3-4 vistas diferenciadas. Dashboard básico. Listados con paginación. Detalle presente. Formularios funcionales.
- **0.1 puntos:** Implementa 2 vistas básicas. Sin dashboard. Sin paginación o limitada. Formularios muy básicos.
- **0 puntos:** Menos de 2 vistas o vistas completamente básicas.

4.4 Paginación (0.5 puntos)

- **0.5 puntos:** Implementa paginación en al menos 2 endpoints que lo requieran. Controles de paginación intuitivos (anterior, siguiente, números de página). Muestra información de paginación (mostrando X-Y de Z resultados). Permite cambiar tamaño de página (10, 25, 50, 100 items). Mantiene estado de paginación en query params para permitir compartir URLs. Maneja correctamente respuestas paginadas del backend (total, page, size, content).
- **0.3 puntos:** Paginación en 2 endpoints. Controles básicos. Información de paginación presente. Tamaño de página fijo o con opciones limitadas.
- **0.1 puntos:** Paginación en 1 endpoint. Controles muy básicos. Poca información de paginación.
- **0 puntos:** Sin paginación o completamente deficiente.

5. Deployment (3 puntos)

- **3 puntos:** Despliegue completo, integral y estable de todo el stack (backend, base de datos, frontend web, frontend mobile vía Expo). Backend desplegado en AWS (ECS/EC2/Elastic Beanstalk) o plataforma cloud equivalente. Base de datos en servicio administrado (RDS, Cloud SQL). Frontend web desplegado (Vercel, Netlify, Amplify, S3+CloudFront). App mobile publicada en Expo o TestFlight/Play Console. Todos los componentes comunicándose correctamente sin errores de CORS. Variables de entorno correctamente configuradas en producción (no hardcodeadas). Secrets y API keys manejados de manera segura. HTTPS configurado en todos los servicios. Autenticación JWT funcionando end-to-end.

Latencia promedio < 500ms en endpoints principales. Disponibilidad continua verificable. Sin vulnerabilidades críticas detectables. URLs públicas accesibles para testing. Dominio propio con HTTPS (bonus reconocido sin impacto negativo si ausente).

- **2 puntos:** Despliegue parcial funcional. Backend y base de datos desplegados y comunicándose. Al menos frontend web O mobile funcional en producción. Algunos warnings menores o disponibilidad intermitente que no comprometen flujo principal. Variables de entorno configuradas. HTTPS en servicios principales. Autenticación funcional. Latencia aceptable. URLs públicas accesibles.
 - **1 punto:** Despliegue básico con problemas. Backend desplegado pero con problemas intermitentes. Base de datos con configuración subóptima. Frontend parcialmente funcional. Errores de CORS ocasionales. Configuración de variables incompleta. HTTPS parcial. Autenticación con problemas. Latencia alta. Disponibilidad inconsistente.
 - **0 puntos:** Sin despliegue funcional. Servicios fuera de línea. Errores de CORS bloqueantes. Base de datos inaccesible. Autenticación no funcional. Sin instancia pública verificable. Cualquier condición que impida uso efectivo desde web o mobile.
-

Estructura Recomendada para la Presentación del Proyecto Front-end

Tiempo total: 20 minutos

La siguiente estructura está diseñada para ayudarles a crear una presentación efectiva en el contexto del curso CS2031 Desarrollo Basado en Plataformas. Pueden ajustar según sus necesidades.

1. Introducción (1 minuto)

Saludo inicial y presentación rápida: No dedicar demasiado tiempo a la presentación de los integrantes, ya que esta información ya es conocida por quienes evalúan.

Gancho inicial: Planteamiento entretenido de una situación o problema real que capte la atención del público y ejemplifique una situación que su proyecto solucione.

Ejemplo: "¿Alguna vez has tenido dificultades para encontrar una aplicación que te ayude a organizar tus tareas diarias de manera eficiente?"

2. Planteamiento del Problema (2 minutos)

Descripción clara y concisa del problema: Explicar por qué este problema es relevante, a quién afecta y cómo le afecta.

Ejemplo: "La mayoría de las aplicaciones de tareas carecen de una integración efectiva con los calendarios personales, lo que causa desorganización y pérdida de productividad."

3. Presentación de la Solución (4 minutos)

Introducción a la aplicación desarrollada: Presentar el nombre y objetivos principales de la aplicación.

Ejemplo: "Les presentamos TaskMaster, una aplicación que integra tus tareas con tu calendario personal de manera intuitiva."

Características principales: Listar las funcionalidades clave de la aplicación, centrándose en lo que hace y no tanto en cómo se implementa. Será necesario el uso de mocks para mostrar partes de la aplicación, ya que la parte de demo cubrirá el funcionamiento de la aplicación real.

Ejemplo: "TaskMaster permite la creación de tareas, la asignación de fechas y horas, y la sincronización automática con Google Calendar."

4. Arquitectura y Tecnologías Utilizadas (2 minutos)

Descripción técnica breve: Explicar cómo está estructurada la aplicación de manera corta y directa, ya que los detalles se evaluarán revisando el repositorio. Acompañado con diagramas para hacer más sencillo de entender y explicar.

Ejemplo: "TaskMaster utiliza React para el frontend, Spring Boot para el backend, PostgreSQL como base de datos y está completamente hosteada en AWS. Adicionalmente, usamos websockets para funcionalidades en tiempo real."

5. Demo en Vivo (6 minutos)

Preparación previa: Tener datos preparados para evitar procesos que tomen mucho tiempo durante la demo. A continuación un ejemplo, si es que tu aplicación no tiene alguna de estas vistas no hay problema es simplemente un ejemplo.

Navegación por la aplicación: Mostrar las pantallas y funcionalidades principales.

- **Login:** Demostrar el proceso de inicio de sesión. El registro es preferible evitarlo por el tiempo que toma, para esto es necesario tener datos ya creados.
- **Dashboard:** Explicar la vista principal y cómo el usuario puede acceder a diferentes secciones. Mostrar las interacciones que pueden generar en los distintos componentes.
- **Funcionalidad principal:** Deben tener preparado un flujo en el cual hacen muestra clara y extensiva de cómo su aplicación soluciona el problema principal.

Interacción y fluidez: Asegurarse de que el demo sea fluido y sin problemas técnicos, para esto es necesario practicar previamente.

6. Ronda de Preguntas (5 minutos)

Preguntas técnicas: Las preguntas se enfocarán en el lado técnico y de implementación del proyecto, incluyendo tanto backend como frontend.

Justificación de tecnologías: Si el grupo utilizó tecnologías adicionales a las tratadas en el curso, deben justificar por qué se eligieron.